

SOCForge - SOC Automation Home Lab

1. Introduction

My project, SOCForge - SOC Automation & Detection Lab, is a home lab designed to simulate a Security Operations Center (SOC) environment. It integrates Wazuh SIEM and Sysmon for endpoint monitoring and threat detection, Shuffle SOAR to automate alert handling, VirusTotal for file-hash threat intelligence enrichment, and TheHive for incident case management. The workflow helps me investigate suspicious activity, enrich alerts with threat intelligence, document incidents, and send email notifications. I built this lab using a Windows 11 virtual machine, VirtualBox, and Vultr cloud instances to gain practical experience with SOC operations and security automation.

2. Basic Terms

What is a Hypervisor?

A hypervisor is software or firmware that allows you to create and run multiple virtual machines on a single physical computer. It allocates hardware resources such as CPU, RAM, storage, and network access to each virtual machine.

There are two main types:

- **Type 1 (Bare-metal):** Runs directly on physical hardware. Examples: VMware ESXi and Microsoft Hyper-V.
- **Type 2 (Hosted):** Runs as an application on an existing operating system. Examples: Oracle VirtualBox and VMware Workstation.

In our project: We use VirtualBox, a Type 2 hypervisor, to run a Windows virtual machine for generating and monitoring endpoint security events without needing a separate physical computer.

What is a Virtual Machine?

A virtual machine is a software-based computer that behaves like a physical computer. It has its own virtual CPU, RAM, storage, network adapter, and operating system. A VM runs on a physical computer through a hypervisor. It is useful for testing software, building labs, and safely isolating different environments.

In our project: The Windows VM acts as the monitored endpoint. We install Sysmon and the Wazuh agent on it so that system activity can be collected and analyzed for suspicious behavior.

What is Windows?

Windows is an operating system developed by Microsoft. It manages computer hardware, applications, users, processes, files, and network connections.

Why are we using Windows?

- It is a common endpoint operating system in enterprise environments.
- It generates security-relevant events, including process creation, logons, and service activity.
- It supports Sysmon and the Wazuh agent.
- It provides an endpoint on which to test detection rules and generate security alerts.

In our project: Windows acts as the endpoint where test activity occurs. Sysmon records selected system events, and the Wazuh agent forwards relevant event data to the Wazuh manager.

What is Sysmon?

Sysmon (System Monitor) is a Windows system service and device driver from Microsoft Sysinternals that records detailed system activity in Windows Event Logs.

Depending on its configuration, Sysmon can record events such as:

- Process creation and command-line arguments.
- Network connections.
- File creation and modification.
- Loading of certain executable images.
- Changes to selected parts of the system.

Sysmon primarily provides telemetry, meaning detailed information about what happened on the endpoint. It does not, by itself, act as a complete SIEM or automatically block threats.

In our project: Sysmon collects endpoint activity. The Wazuh agent reads the relevant event logs and sends the data to Wazuh for analysis and alert generation.

What is Vultr Cloud and why are we using this in this project?

Vultr is a cloud infrastructure provider that offers virtual servers, networking, storage, and other cloud services.

Instead of purchasing physical servers, you can rent cloud-based virtual machines and access them remotely.

In our project: Vultr hosts the cloud servers used for services such as Wazuh and TheHive. This allows the Windows endpoint running on our local computer to communicate with security services hosted remotely.

Benefits include:

- No need to purchase dedicated server hardware.
- Remote access to the lab's security infrastructure.
- Separate servers for different services.
- The ability to choose server resources based on the requirements of each service.

What is Wazuh?

Wazuh is an open-source security platform used for endpoint monitoring, log analysis, threat detection, file integrity monitoring, vulnerability detection, and security compliance.

Its main components include:

- Wazuh agent: Installed on an endpoint to collect security data.
- Wazuh server/manager: Receives and analyzes data, applies decoders and detection rules, and generates alerts.
- Wazuh indexer: Stores and indexes alert and event data for searching and analysis.
- Wazuh dashboard: Provides a graphical interface for investigating alerts and monitoring security activity.

In our project: The Wazuh agent collects relevant Windows event data. The Wazuh manager evaluates that data against detection rules, including our custom rule, and generates an alert when the configured conditions are met. The alert can then be sent to Shuffle for further processing.

What is SOAR?

SOAR stands for Security Orchestration, Automation, and Response.

It is a category of security technology that connects different security tools and automates repetitive tasks in an incident-handling workflow.

- Orchestration: Connects different tools and services.
- Automation: Executes predefined tasks without requiring manual action at every step.
- Response: Helps analysts handle alerts and incidents through defined workflows.

For example, a SOAR workflow can receive an SIEM alert, extract a file hash, query a threat-intelligence service, create a case, and notify an analyst.

In our project: Shuffle acts as the SOAR platform. It receives an alert from Wazuh and coordinates the subsequent workflow, including VirusTotal enrichment, TheHive case management, and email notification.

What is XDR?

XDR stands for Extended Detection and Response.

XDR is a security approach that collects and correlates security data from multiple sources, such as endpoints, networks, email systems, cloud workloads, and identity services. It helps analysts investigate threats that may span several parts of an organization's environment.

Example: An employee receives a malicious email, opens an attachment, and the attachment starts a suspicious process. XDR can correlate email, endpoint, and network activity to help reveal the broader attack.

In our project: Our lab demonstrates parts of an integrated detection and response workflow, but it is not a full XDR deployment. It focuses on Windows endpoint telemetry, Wazuh detection, Shuffle automation, threat-intelligence enrichment, and case management.

What is EDR?

EDR stands for Endpoint Detection and Response.

EDR technology monitors endpoint devices such as laptops, desktops, and servers. It collects endpoint activity, detects suspicious behavior, and supports investigation and response.

Typical EDR capabilities include:

- Process and command-line monitoring.
- Detection of suspicious endpoint behavior.
- Investigation of process activity and related events.
- Response actions, such as isolating an endpoint or terminating a process, depending on the product.

In our project: Sysmon provides detailed Windows telemetry, and the Wazuh agent collects and forwards relevant events. Wazuh provides endpoint security monitoring and detection capabilities, although our setup should not be described as a complete commercial EDR solution with all the same response features. Also, Sysmon is a telemetry collection tool, not a complete EDR by itself.

What is SIEM?

SIEM stands for Security Information and Event Management.

A SIEM collects security logs and events from multiple systems, analyzes them using rules and other detection methods, and generates alerts to help security analysts identify suspicious activity.

Its main functions include:

- Log collection and centralization.
- Event analysis and correlation.
- Alert generation.
- Searching and investigating security events.
- Reporting and compliance support.

In our project: Wazuh acts as our SIEM platform. It receives Windows event data through the Wazuh agent, evaluates the events using detection rules, and displays generated alerts in its dashboard

What is Uncomplicated Firewall (UFW)?

UFW stands for Uncomplicated Firewall. It is a command-line firewall management tool commonly used on Ubuntu Linux to configure the system's firewall. It provides a simpler interface for managing network traffic rules, typically through the Linux firewall framework.

For example:

```
sudo ufw status
```

Displays the current firewall status and configured rules.

```
sudo ufw allow 22/tcp
```

Allows incoming TCP traffic on port 22, commonly used for SSH.

```
sudo ufw deny 8080/tcp
```

Blocks incoming TCP traffic on port 8080.

In our project: UFW can be used on the Vultr Ubuntu servers to restrict access to services such as SSH, Wazuh, and TheHive. Only the necessary ports should be allowed, ideally from trusted IP addresses where possible.

Important: Allowing a port in UFW does not guarantee that the service is secure. Cloud firewall rules, service configuration, authentication, and application-level access controls also matter.

What is Cassandra?

Apache Cassandra is an open-source, distributed NoSQL database designed to store large amounts of data across multiple nodes while supporting high availability and scalability. Unlike a traditional relational database, Cassandra stores data using a distributed data model designed for workloads that need to handle large volumes of writes and remain available across nodes.

In our project: Cassandra is described in the lab documentation as a supporting database for TheHive. It stores application data required by compatible TheHive deployments.

Technical note: Cassandra and Elasticsearch serve different purposes. Cassandra is a database for application data, while Elasticsearch is primarily a search and analytics engine. Before deployment, verify that the specific TheHive release supports the database and search-engine versions you intend to use.

What is Elasticsearch?

Elasticsearch is a distributed search and analytics engine that stores, indexes, and searches large volumes of data. It is commonly used to search logs, application events, and other structured or semi-structured information. Instead of reading every log file manually, users can search indexed data using fields, filters, and queries.

In our project: Elasticsearch is relevant to the search and storage architecture described for TheHive's supporting services in the lab documentation. It is used to support efficient indexing and searching of data in compatible deployments.

Important technical distinction: The Wazuh stack uses the Wazuh indexer, which is based on OpenSearch, rather than Elasticsearch. TheHive's search and storage requirements vary by version, so the exact Elasticsearch compatibility must be checked against the TheHive version being installed.

What is TheHive?

TheHive is a security incident response and case management platform used by SOC teams and incident responders to investigate, organize, and track security incidents.

Its capabilities include:

- Creating and managing cases.
- Recording alerts, observables, and investigation notes.
- Assigning tasks to analysts.
- Tracking investigation progress.
- Integrating with other security tools through APIs and connectors.

An observable is an item of interest associated with an investigation, such as an IP address, domain, URL, email address, or file hash.

In our project: Shuffle sends relevant alert information to TheHive to create or populate an incident case, depending on the workflow configuration. This gives analysts a place to review and track the alert instead of relying only on the SIEM dashboard or email.

What is VirusTotal?

VirusTotal is a threat-intelligence and analysis service that helps users investigate suspicious files, URLs, domains, and IP addresses. It aggregates analysis and reputation information from multiple security engines and other sources. For file investigations, a common approach is to calculate the file's SHA-256 hash and search for that hash in VirusTotal. A hash is a fixed-length value calculated from data. It acts as a digital fingerprint of the file. SHA-256 produces a 256-bit hash, normally represented as 64 hexadecimal characters.

In our project: Shuffle extracts the relevant file hash from the alert or workflow input and queries the VirusTotal API. The returned information can help an analyst understand whether the file has previously been reported as suspicious or malicious.

What is Mimikatz?

Mimikatz is a security research and penetration-testing tool commonly associated with extracting credentials and other authentication material from Windows systems. It can also be used to demonstrate and test certain Windows authentication weaknesses and credential-access techniques. Attackers may misuse Mimikatz for credential theft and lateral movement. Security teams use controlled simulations and detection rules to understand and identify related activity.

In our project: Mimikatz is the test tool used to generate security-relevant activity in the Windows lab. The objective is to check whether endpoint events are collected and whether Wazuh generates an alert when the activity matches the configured detection rule.

What is Shuffle?

Shuffle is a SOAR platform that allows security teams to build workflows connecting different tools and services. It provides a visual workflow builder and integrations for automating security tasks. A workflow can contain triggers, conditions, API requests, and actions. For example, an incoming alert can trigger a VirusTotal lookup, followed by a case-creation action in TheHive and an email notification.

In our project: Shuffle receives alerts from Wazuh through a webhook and orchestrates the subsequent steps:

- Receive the Wazuh alert.
- Extract the relevant file hash or other alert fields.
- Query VirusTotal for threat-intelligence information.
- Send the relevant incident data to TheHive.
- Send an email notification to the analyst.

3. Prerequisites

- Host Machine with sufficient RAM (at least 8 GB recommended) and disk space (at least 50 GB free).
- Internet Connection for downloading software and updates.
- Basic Linux Command Line Knowledge (to install packages, edit configuration, files, etc.).
- Virtual box (Hypervisor)
- Windows 11 (Virtual Machine)

4. Architecture



5. Sysmon Installation

- To install Sysmon, run Windows 10 on VirtualBox. Open Microsoft Edge in the VM and go to this website: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
- Download Sysmon in a zip file format and extract it once it has been downloaded
- Go to <https://raw.githubusercontent.com/olafhartong/sysmon-modular/refs/heads/master/sysmonconfig.xml> and download sysmonconfig.xml
- Open Powershell as an administrator and run the below commands to install Sysmon:

`cd <path of extracted Sysmon file>`

`.\Sysmon64.exe -i .\sysmonconfig.xml`

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> cd C:\Users\Sue\Downloads\Sysmon
PS C:\Users\Sue\Downloads\Sysmon> pwd

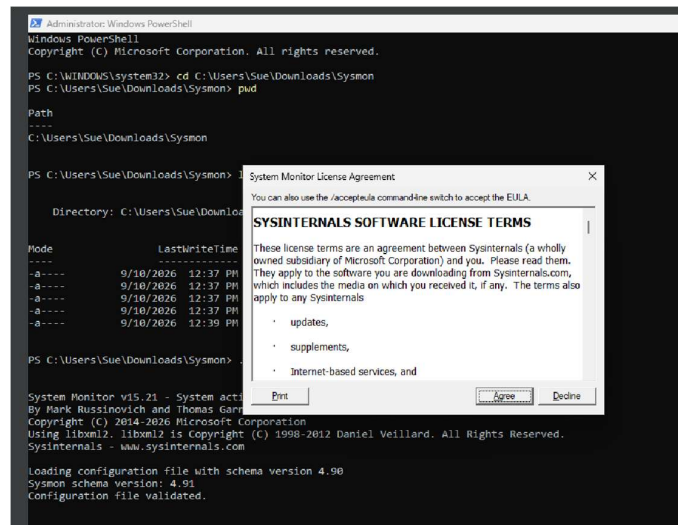
Path
----
C:\Users\Sue\Downloads\Sysmon

PS C:\Users\Sue\Downloads\Sysmon> ls

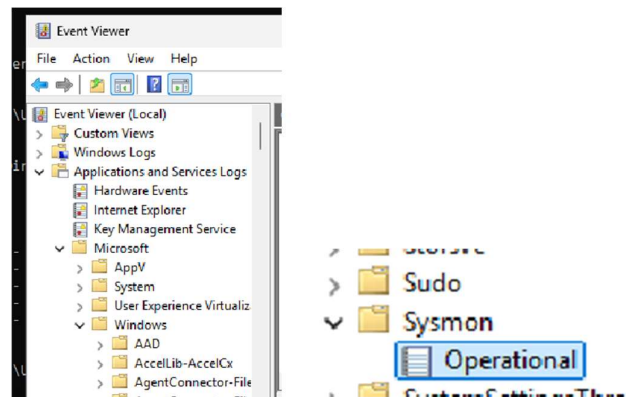
Directory: C:\Users\Sue\Downloads\Sysmon

Mode                LastWriteTime         Length Name
----                -
-a----          9/10/2026 12:37 PM             253169 sysmonconfig.xml

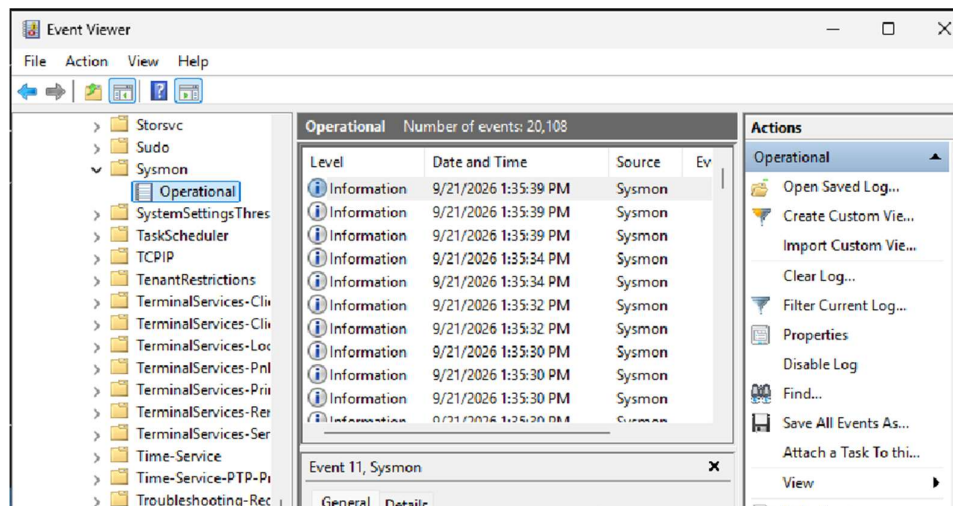
PS C:\Users\Sue\Downloads\Sysmon> .\Sysmon64.exe -i .\sysmonconfig.xml
```



- To check if Sysmon is installed correctly and is running: Event viewer -> Applications and Services -> Microsoft -> Windows -> Sysmon -> double click on Operational



- If Sysmon is running correctly then we will be able to view some logs



6. Wazuh Installation and Configuration

- To install Wazuh, we will require a cloud provider. For that we will use Vultr. Navigate to this website: <https://www.vultr.com>
- After Sign-in, create an instance named SOC-Automation-Wazuh

Instances Documentation API

Q Search instances... Create Instance +

Name / UUID	Location	Image	IP / Connectivity	Created	Status	Charges	...	
SOC-Automation-Wazuh	Mumbai, IN (APAC)	Ubuntu 24.04 LTS x64	Public: IPv4	2026-09-20 22:57:03	Running	\$0.08	...	

Showing 1 entry 10 / page

- Open Windows 11 VM on VirtualBox
- Open Powershell and type the given command to connect to SOC-Automation-Wazuh instance : `ssh root@<ip address of the instance>`

```
PS C:\Users\Sue\Downloads\Sysmon> ssh root@
The authenticity of host '132.145.126.101 (132.145.126.101)' can't be established.
key fingerprint is SHA256:
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '132.145.126.101' (SHA256:
root@132.145.126.101's password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)
```

- To download Wazuh, type the given command on powershell: `curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a`

```
root@SOC-Automation-Wazuh:~# curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-in
stall.sh -a
11/09/2026 13:03:03 INFO: Starting Wazuh installation assistant. Wazuh version: 4.14.7
11/09/2026 13:03:03 INFO: Verbose logging redirected to /var/log/wazuh-install.log
11/09/2026 13:03:40 INFO: Verifying that your system meets the recommended minimum hardware requirements.
11/09/2026 13:03:40 INFO: Wazuh web interface port will be 443.
11/09/2026 13:03:42 WARNING: The system has UFW enabled. Please ensure that traffic is allowed on these ports:
1515, 1516, 443.
11/09/2026 13:04:11 INFO: --- Dependencies ---
11/09/2026 13:04:11 INFO: Installing apt-transport-https.
11/09/2026 13:04:16 INFO: Installing debhelper.
11/09/2026 13:04:48 INFO: Wazuh repository added.
11/09/2026 13:04:48 INFO: --- Configuration files ---
11/09/2026 13:04:48 INFO: Generating configuration files.
11/09/2026 13:04:48 INFO: Generating the root certificate.
11/09/2026 13:04:49 INFO: Generating admin certificates.
11/09/2026 13:04:49 INFO: Generating Wazuh indexer certificates.
11/09/2026 13:04:49 INFO: Generating Filebeat certificates.
11/09/2026 13:04:59 INFO: Generating Wazuh dashboard certificates.
11/09/2026 13:04:59 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, an
d passwords necessary for installation.
11/09/2026 13:04:51 INFO: --- Wazuh indexer ---
11/09/2026 13:04:51 INFO: Starting Wazuh indexer installation.
11/09/2026 13:05:22 INFO: Wazuh indexer installation finished.
11/09/2026 13:05:22 INFO: Wazuh indexer post-install configuration finished.
11/09/2026 13:05:22 INFO: Starting service wazuh-indexer.
11/09/2026 13:05:45 INFO: wazuh-indexer service started.
11/09/2026 13:05:45 INFO: Initializing Wazuh indexer cluster security settings.
11/09/2026 13:05:51 INFO: Wazuh indexer cluster security configuration initialized.
11/09/2026 13:05:51 INFO: Wazuh indexer cluster initialized.
11/09/2026 13:05:51 INFO: --- Wazuh server ---
11/09/2026 13:05:51 INFO: Starting the Wazuh manager installation.
```

- Note down the username and password displayed at the end of the installation

```
root@SOC-Automation-Wazu:~# systemctl start wazuh-indexer
11/09/2026 13:05:22 INFO: Starting service wazuh-indexer.
11/09/2026 13:05:45 INFO: wazuh-indexer service started.
11/09/2026 13:05:45 INFO: Initializing Wazuh indexer cluster security settings.
11/09/2026 13:05:51 INFO: Wazuh indexer cluster security configuration initialized.
11/09/2026 13:05:51 INFO: Wazuh indexer cluster initialized.
11/09/2026 13:05:51 INFO: --- Wazuh server ---
11/09/2026 13:05:51 INFO: Starting the Wazuh manager installation.
11/09/2026 13:07:16 INFO: Wazuh manager installation finished.
11/09/2026 13:07:16 INFO: Wazuh manager vulnerability detection configuration finished.
11/09/2026 13:07:16 INFO: Starting service wazuh-manager.
11/09/2026 13:07:39 INFO: wazuh-manager service started.
11/09/2026 13:07:39 INFO: Starting Filebeat installation.
11/09/2026 13:07:50 INFO: Filebeat installation finished.
11/09/2026 13:07:50 INFO: Filebeat post-install configuration finished.
11/09/2026 13:07:50 INFO: Starting service filebeat.
11/09/2026 13:07:52 INFO: Filebeat service started.
11/09/2026 13:07:52 INFO: --- Wazuh dashboard ---
11/09/2026 13:07:52 INFO: Starting Wazuh dashboard installation.
11/09/2026 13:08:55 INFO: Wazuh dashboard installation finished.
11/09/2026 13:08:55 INFO: Wazuh dashboard post-install configuration finished.
11/09/2026 13:08:55 INFO: Starting service wazuh-dashboard.
11/09/2026 13:08:56 INFO: wazuh-dashboard service started.
11/09/2026 13:08:59 INFO: Updating the internal users.
11/09/2026 13:09:05 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/internaluser
e-backup folder.
11/09/2026 13:09:20 INFO: The filebeat.yml file has been updated to use the Filebeat Keystore username and pas
sword.
11/09/2026 13:10:23 INFO: Initializing Wazuh dashboard web application.
11/09/2026 13:10:23 INFO: Wazuh dashboard web application initialized.
11/09/2026 13:10:23 INFO: --- Summary ---
11/09/2026 13:10:23 INFO: You can access the web interface https://wazuh-dashboard-ip:443
Users: admin
Password:
11/09/2026 13:10:23 INFO: Installation finished.
root@SOC-Automation-Wazu:~#
```

- To start Wazuh, type the given below commands:

- `systemctl status wazuh-manager`
- `systemctl start wazuh-manager`

```
root@SOC-Automation-Wazu:~# systemctl start wazuh-manager
root@SOC-Automation-Wazu:~# systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-21 09:55:53 UTC; 2h 38min ago
     Tasks: 174 (limit: 9385)
    Memory: 4.96 (peak: 5.06)
       CPU: 2min 8.303s
    CGroup: /system.slice/wazuh-manager.service
           └─50514 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─50515 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─50516 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─50519 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
           └─50522 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
```

- `systemctl status wazuh-dashboard`
- `systemctl start wazuh-dashboard`

```
root@SOC-Automation-Wazu:~# systemctl start wazuh-dashboard
root@SOC-Automation-Wazu:~# systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-21 09:55:55 UTC; 2h 38min ago
     Main PID: (node)
     Tasks: 11 (limit: 9385)
    Memory: 196.1M (peak: 305.2M)
       CPU: 26.021s
    CGroup: /system.slice/wazuh-dashboard.service
```

- `systemctl status wazuh-indexer`
- `systemctl start wazuh-indexer`

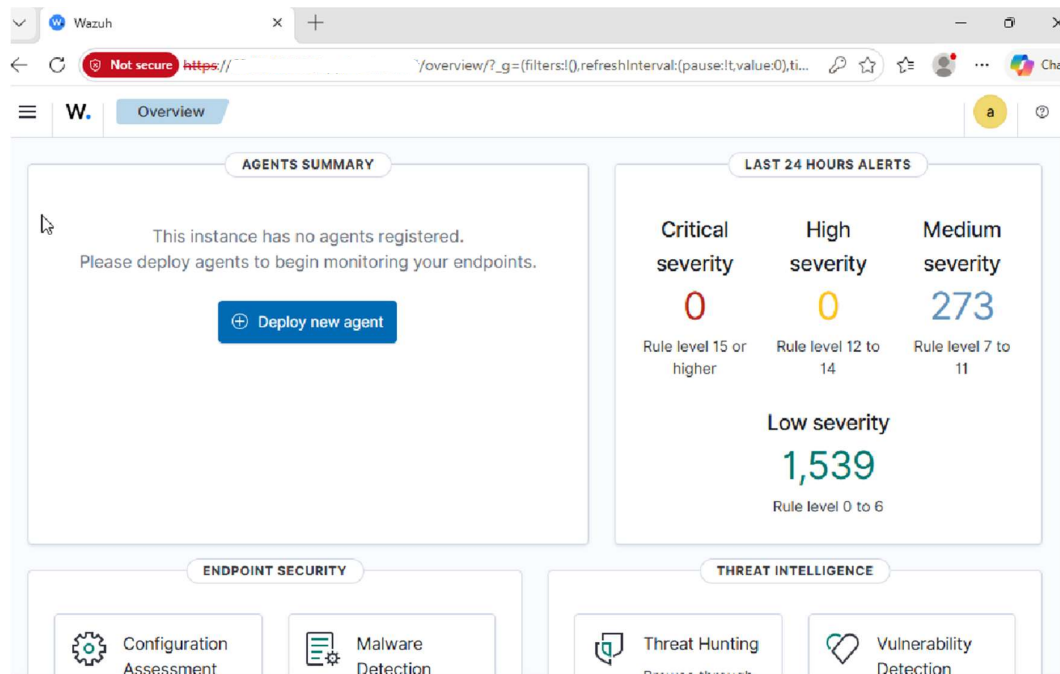
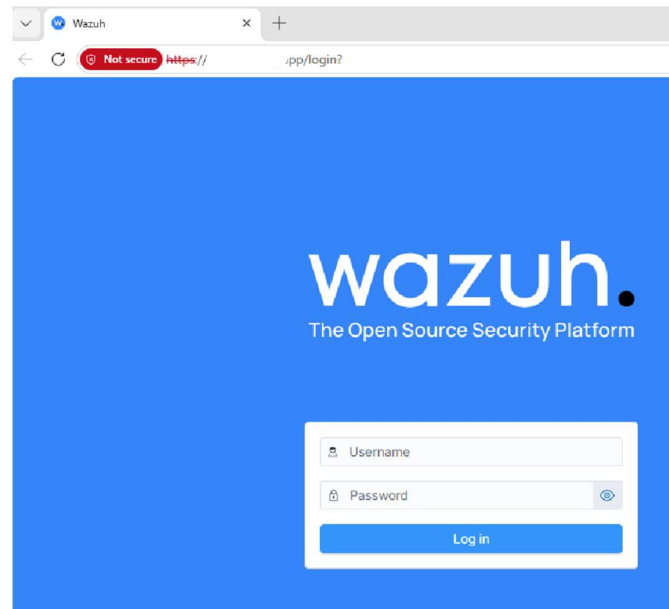
```
root@SOC-Automation-Wazu:~# systemctl start wazuh-indexer
root@SOC-Automation-Wazu:~# systemctl status wazuh-indexer
● wazuh-indexer.service - wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-21 09:55:10 UTC; 2h 39min ago
     Docs: https://documentation.wazuh.com
     Main PID: 1 (java)
     Tasks: 80 (limit: 9385)
    Memory: 1.5G (peak: 1.5G)
       CPU: 3min 26.582s
    CGroup: /system.slice/wazuh-indexer.service
           └─1

Sep 21 09:54:54 SOC-Automation-Wazu systemd-entrypoint[57971]: WARNING: System::setSecurityManager has
Sep 21 09:54:54 SOC-Automation-Wazu systemd-entrypoint[57971]: WARNING: Please consider reporting this
Sep 21 09:54:54 SOC-Automation-Wazu systemd-entrypoint[57971]: WARNING: System::setSecurityManager wi
Sep 21 09:54:54 SOC-Automation-Wazu systemd-entrypoint[57971]: Sep 21, 2026 9:54:55 AM sun.util.loca
Sep 21 09:54:55 SOC-Automation-Wazu systemd-entrypoint[57971]: WARNING: COMPAT locale provider will b
Sep 21 09:54:55 SOC-Automation-Wazu systemd-entrypoint[57971]: WARNING: A terminally deprecated metho
Sep 21 09:54:55 SOC-Automation-Wazu systemd-entrypoint[57971]: WARNING: System::setSecurityManager has
Sep 21 09:54:55 SOC-Automation-Wazu systemd-entrypoint[57971]: WARNING: Please consider reporting this
Sep 21 09:54:55 SOC-Automation-Wazu systemd-entrypoint[57971]: WARNING: System::setSecurityManager wi
Sep 21 09:55:10 SOC-Automation-Wazu systemd[1]: Started wazuh-indexer.service - wazuh-indexer.
```

- `ufw allow 443`

```
root@SOC-Automation-Wazuh:~# ufw allow 443
Rule added
Rule added (v6)
root@SOC-Automation-Wazuh:~#
```

- Open Microsoft edge and navigate to wazuh: <https://<SOC-Automation-Wazuh Instance ip>>



7. TheHive Installation and Configuration

- To install Wazuh, we will require a cloud provider. For that we will use Vultr. Navigate to this website: <https://www.vultr.com>
- After Sign-in, create an instance named SOC-Automation-TheHive

Instances								Documentation	API
Search instances...								Create Instance +	
Name / UUID ↓	Location ↓	Image ↓	IP / Connectivity ↓	Created ↓	Status ↓	Charges ↓	...		
SOC-Automation-Wazuh	 Mum..., IN (APAC)	 Ubuntu 24.04 LTS x64	Public: IPv4	2026-09-20 22:57:03	 Running	\$0.14	...		
SOC-Automation-TheHive	 Mum..., IN (APAC)	 Ubuntu 24.04 LTS x64	Public: IPv4	2026-09-21 00:10:50	 Running	\$0.12	...		

- To download hive, open a powershell on window 10 VM and connect to SOC-Automation-TheHive: `ssh root@<public ip address of the instance>`

```
PS C:\WINDOWS\system32> ssh root@10.10.10.10
The authenticity of host '10.10.10.10 (10.10.10.10)' can't be established.
ECDSA key fingerprint is SHA256:10.10.10.10.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.10.10' (10.10.10.10) to the list of known hosts
root@10.10.10.10's password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)
```

7.1 Install required dependencies

- Start by installing the necessary dependencies for TheHive. Run the following commands:

```
sudo apt update
```

```
sudo apt install wget curl gnupg coreutils apt-transport-https git ca-
certificates ca-certificates-java software-properties-common python3-pip lsb-
release unzip
```

```
root@SOC-Automation-THelvis:~# sudo apt update
sudo apt install wget curl gnupg coreutils apt-transport-https git ca-certificates ca-certificates-java software-properties-common python3-pip lsb-release unzip
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://ubuntu.mirror.constant.com noble InRelease
Get:3 http://ubuntu.mirror.constant.com noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,020 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,217 kB]
Hit:6 http://ubuntu.mirror.constant.com noble-backports InRelease
Get:7 http://ubuntu.mirror.constant.com noble-updates/universe amd64 Packages [1,697 kB]
Get:8 http://ubuntu.mirror.constant.com noble-updates/universe Translation-en [340 kB]
Fetched 4,526 kB in 4s (1,106 kB/s)
Reading package lists... Done
Building dependency tree... Done
```

7.2 Set up the Java virtual machine (JVM)

- TheHive requires Java to run its application server and to manage various processes. Only Java 11 is supported. Run the following commands:

```
wget -qO- https://apt.corretto.aws/corretto.key | sudo gpg --dearmor -o /usr/share/keyrings/corretto.gpg
```

```

echo "deb [signed-by=/usr/share/keyrings/corretto.gpg]
https://apt.corretto.aws stable main" | sudo tee -a
/etc/apt/sources.list.d/corretto.sources.list

sudo apt update

sudo apt install java-common java-11-amazon-corretto-jdk

echo JAVA_HOME="/usr/lib/jvm/java-11-amazon-corretto" | sudo tee -a
/etc/environment

export JAVA_HOME="/usr/lib/jvm/java-11-amazon-corretto"

```

```

root@SOC-Automation-TheHive:~# wget -qO- https://apt.corretto.aws/corretto.key | sudo gpg --dearmor -o /usr/share/keyrings/corretto.gpg
echo "deb [signed-by=/usr/share/keyrings/corretto.gpg] https://apt.corretto.aws stable main" | sudo tee -a /etc/apt/sources.list.d/corretto.sources.list
sudo apt update
sudo apt install java-common java-11-amazon-corretto-jdk
echo JAVA_HOME="/usr/lib/jvm/java-11-amazon-corretto" | sudo tee -a /etc/environment
export JAVA_HOME="/usr/lib/jvm/java-11-amazon-corretto"
deb [signed-by=/usr/share/keyrings/corretto.gpg] https://apt.corretto.aws stable main
Get:1 https://apt.corretto.aws stable InRelease [10.7 kB]
Get:2 https://apt.corretto.aws stable/main amd64 Packages [28.8 kB]
Hit:3 http://ubuntu.mirror.constant.com noble InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 http://ubuntu.mirror.constant.com noble-updates InRelease
Hit:6 http://ubuntu.mirror.constant.com noble-backports InRelease
Fetched 39.5 kB in 1s (31.5 kB/s)

```

- Verify the installation.

```
java -version
```

```

root@SOC-Automation-TheHive:~# java -version
openjdk version "11.0.32.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-11.0.32.10.1 (build 11.0.32.1+10-LTS)
OpenJDK 64-Bit Server VM Corretto-11.0.32.10.1 (build 11.0.32.1+10-LTS, mixed mode)
root@SOC-Automation-TheHive:~#

```

7.3 Install and configure Apache Cassandra

Apache Cassandra is a database system that's used in TheHive as the back-end database for storing and managing incident response data.

7.3.1 Install Cassandra

- Download Cassandra repository keys.

```
wget -qO- https://downloads.apache.org/cassandra/KEYS | sudo gpg --dearmor -o /usr/share/keyrings/cassandra-archive.gpg
```

- Check if the /etc/apt/sources.list.d/cassandra.sources.list file exists. If it doesn't, create it.

```

echo "deb [signed-by=/usr/share/keyrings/cassandra-archive.gpg]
https://deb.debian.org/debian/cassandra 41x main" | sudo tee -a
/etc/apt/sources.list.d/cassandra.sources.list

```

- Update your package index and install Cassandra using the following commands:

```
sudo apt update
```

```
sudo apt install Cassandra
```

```
root@SOC-Automation-TheHive:~# wget -qO - https://downloads.apache.org/cassandra/KEYS | sudo gpg --dearmor -o /usr/share/keyrings/cassandra-archive.gpg
root@SOC-Automation-TheHive:~# sudo cat /etc/apt/sources.list.d/cassandra.sources.list
cat: /etc/apt/sources.list.d/cassandra.sources.list: No such file or directory
root@SOC-Automation-TheHive:~# echo "deb [signed-by=/usr/share/keyrings/cassandra-archive.gpg] https://deb.debian.org/debian/cassandra-archive.org 4ix main" | sudo tee -a /etc/apt/sources.list.d/cassandra.sources.list
deb [signed-by=/usr/share/keyrings/cassandra-archive.gpg] https://deb.debian.org/debian/cassandra-archive.org 4ix main
root@SOC-Automation-TheHive:~# sudo apt update
sudo apt install cassandra
Hit:1 https://apt.corretto.aws stable InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:3 http://ubuntu.mirror.constant.com noble InRelease
Hit:4 http://ubuntu.mirror.constant.com noble-updates InRelease
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,020 kB]
Hit:7 http://ubuntu.mirror.constant.com noble-backports InRelease
```

- Verify that Cassandra is installed.

```
sudo systemctl status cassandra
```

- This will show you the status of the Cassandra service. If Cassandra is installed you should see details about the service. If it's not installed, you will receive an error indicating that the service isn't found. By default, data is stored in `/var/lib/cassandra`. Set appropriate permissions for this directory to avoid any issues with data storage and access.

```
sudo chown -R cassandra:cassandra /var/lib/cassandra
```

```
root@SOC-Automation-TheHive:~# sudo systemctl status cassandra
● cassandra.service - LSB: distributed storage system for structured data
   Loaded: loaded (/etc/init.d/cassandra; generated)
   Active: active (running) since Mon 2026-09-21 15:34:52 UTC; 3min 53s ago
     Docs: man:systemd-sysv-generator(8)
   Process: ExecStart=/etc/init.d/cassandra start (code=exited, status=0/SUCCESS)
    Tasks: 52 (limit: 19037)
   Memory: 4.3G (peak: 4.3G)
      CPU: 25.253s
   CGroup: /system.slice/cassandra.service
           └─5274 /usr/bin/java

Sep 21 15:34:52 SOC-Automation-TheHive systemd[1]: Starting cassandra.service - LSB: distributed storage system for structured data.
Sep 21 15:34:52 SOC-Automation-TheHive systemd[1]: Started cassandra.service - LSB: distributed storage system for structured data.
root@SOC-Automation-TheHive:~# sudo chown -R cassandra:cassandra /var/lib/cassandra
```

7.3.2 Configure JVM options for Cassandra

The Java virtual machine (JVM) is what runs Cassandra. The JVM options control how much memory Cassandra can use, how it manages that memory, and other performance-related settings. By default, Java determines heap size automatically, which isn't recommended for production environments and may cause memory contention or out-of-memory errors.

- Check if the `/etc/cassandra/jvm-server.options` exists. If it doesn't, create it.
- Open the `/etc/cassandra/jvm-server.options` file using a text editor.
- In the `jvm-server.options` file, set the JVM options.

```
root@SOC-Automation-TheHive:~# sudo nano /etc/cassandra/jvm-server.options
```

Heap size guidelines for Cassandra

Heap allocation must not exceed 50% of the available RAM and should not be less than 2 GB. Available RAM refers to the memory remaining after accounting for the operating system and other services running on the same host.

-Xms<heap_size>

-Xmx<heap_size>

-Xmn<young_gen_size>

Replace <heap_size> with the desired heap size and <young_gen_size> with the desired young generation size. Xms sets the initial heap size, Xmx sets the maximum, and Xmn sets the size of the young generation, where short-lived objects are allocated.

- Save your modifications in the jvm-server.options file.

```
# Heap size is automatically calculated by cassandra
# formula: max(min(1/2 ram, 1024MB), min(1/4 ram, 8192MB))
# That is:
# - calculate 1/2 ram and cap to 1024MB
# - calculate 1/4 ram and cap to 8192MB
# - pick the max
#
# For production use you may wish to adjust this
# If that's the case, uncomment the -Xmx and Xms
# automatic calculation of JVM heap memory.
#
# It is recommended to set min (-Xms) and max (-Xmx)
# the same value to avoid stop-the-world GC pauses
# so that we can lock the heap in memory on startup
# of it from being swapped out.
-Xms2G
-Xmx4G

# Young generation size is automatically calculated
# based on this formula: min(100 * num_cores, 1/4 ram)
#
# The main trade-off for the young generation is
# is, the longer GC pause times will be. The shorter
# expensive GC will be (usually).
#
# It is not recommended to set the young generation
# G1 GC, since that will override the target pause
# More info: http://www.oracle.com/technetwork/articles/javase/g1gc-142429.html
#
# The example below assumes a modern 8-core+ machine
# times. If in doubt, and if you do not particularly
# 100 MB per physical CPU core.
-Xmn200M
```

7.3.3 Configure Cassandra

Configure Cassandra by modifying settings within the cassandra.yaml file.

- Locate the cassandra.yaml file.

```
root@SOC-Automation-TheHive:~# sudo find / -name "cassandra.yaml" 2>/dev/null
/etc/cassandra/cassandra.yaml
```

- Open the cassandra.yaml file using a text editor.

- In the `cassandra.yaml` file, set the `cluster_name` parameter to the desired name. This name will help identify your Cassandra cluster. Replace Test Cluster with your new cluster name.
- Set the appropriate permissions on each directory path to ensure Cassandra can read and write data properly.

`sudo chown -R cassandra:cassandra /etc/cassandra/cassandra.yaml`

```
root@SOC-Automation-TheHive:~# sudo nano /etc/cassandra/cassandra.yaml
root@SOC-Automation-TheHive:~# sudo chown -R cassandra:cassandra /etc/cassandra/cassandra.yaml
root@SOC-Automation-TheHive:~#
```

7.3.4 Start the Cassandra service

- Check whether the Cassandra service started automatically before configuring it.

`sudo systemctl status cassandra`

- If it's running, stop it and remove existing data.

`sudo systemctl stop cassandra`

`sudo rm -rf /var/lib/cassandra/*`

- Start the Cassandra service.

`sudo systemctl start cassandra`

- Enable the Cassandra service to restart automatically after a system reboot.

`sudo systemctl enable cassandra`

- Verify that Cassandra is running.

`sudo systemctl status cassandra`

```
root@SOC-Automation-TheHive:~# sudo systemctl status cassandra
● cassandra.service - LSB: distributed storage system for structured data
   Loaded: loaded (/etc/init.d/cassandra; generated)
   Active: active (running) since Mon 2026-09-21 15:34:52 UTC; 23min ago
     Docs: man:systemd-sysv-generator(8)
  Process: 5129 ExecStart=/etc/init.d/cassandra start (code=exited, status=0/SUCCESS)
    Tasks: 58 (limit: 19037)
   Memory: 4.3G (peak: 4.3G)
      CPU: 39.946s
   CGroup: /system.slice/cassandra.service
           └─5274 /usr/bin/java -ea -da:net.openhft... -XX:+UseThreadPriorities -XX:+HeapDumpOnOutOfMemoryError -Xss256k -X

Sep 21 15:34:52 SOC-Automation-TheHive systemd[1]: Starting cassandra.service - LSB: distributed storage system for structured data
Sep 21 15:34:52 SOC-Automation-TheHive systemd[1]: Started cassandra.service - LSB: distributed storage system for structured data

root@SOC-Automation-TheHive:~# sudo systemctl stop cassandra
root@SOC-Automation-TheHive:~# sudo rm -rf /var/lib/cassandra/*
root@SOC-Automation-TheHive:~# sudo systemctl start cassandra
root@SOC-Automation-TheHive:~# sudo systemctl enable cassandra
cassandra.service is not a native service, redirecting to systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable cassandra
root@SOC-Automation-TheHive:~# sudo systemctl status cassandra
● cassandra.service - LSB: distributed storage system for structured data
   Loaded: loaded (/etc/init.d/cassandra; generated)
   Active: active (running) since Mon 2026-09-21 15:58:52 UTC; 27s ago
     Docs: man:systemd-sysv-generator(8)
    Tasks: 51 (limit: 19037)
   Memory: 2.3G (peak: 2.3G)
      CPU: 14.745s
   CGroup: /system.slice/cassandra.service
           └─5633 /usr/bin/java
```

7.4 Install and configure Elasticsearch

Elasticsearch is a data indexing and search engine that's used in TheHive to manage data indices. Starting with version 5.5, it can also replace Apache Cassandra (JanusGraph) for storing audit logs.

7.4.1 Install Elasticsearch

- Add Elasticsearch repository references and download Elasticsearch repository keys.

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg -  
-dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
```

```
sudo apt-get install apt-transport-https
```

```
root@SOC-Automation-TheHive:~# wget -qO - https://arti  
facts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --d  
earmor -o /usr/share/keyrings/elasticsearch-keyring.gp  
g  
sudo apt-get install apt-transport-https  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
apt-transport-https is already the newest version (2.8  
.3).  
0 upgraded, 0 newly installed, 0 to remove and 5 not u  
pgraded.  
root@SOC-Automation-TheHive:~#
```

- Check if the `/etc/apt/sources.list.d/elastic-8.x.list` file exists. If it doesn't, create it.
- Add the repository to your system by appending the following line to the `/etc/apt/sources.list.d/elastic-8.x.list` file.

```
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg]  
https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee  
/etc/apt/sources.list.d/elastic-8.x.list
```

- Update your package index and install Elasticsearch using the following commands:

```
sudo apt update
```

```
sudo apt install elasticsearch
```

7.4.2 Configure Elasticsearch

- Configure the `/etc/elasticsearch/elasticsearch.yml` file
- Open the `/etc/elasticsearch/elasticsearch.yml` file using a text editor.

- In the elasticsearch.yml file, set the cluster.name parameter to the desired name. This name will help identify your Elasticsearch cluster.
- Replace my-application with your new cluster name.

7.4.3 Start the Elasticsearch service

- Check whether the Elasticsearch service started automatically before configuring it.

```
sudo systemctl status elasticsearch
```

- If it's running, stop it and remove existing data.

```
sudo systemctl stop elasticsearch
```

```
sudo rm -rf /var/lib/elasticsearch/*
```

- Start the Elasticsearch service.

```
sudo systemctl start elasticsearch
```

- Enable the Elasticsearch service to restart automatically after a system reboot.

```
sudo systemctl enable elasticsearch
```

- Verify that Elasticsearch is running.

```
sudo systemctl status elasticsearch
```

- If Elasticsearch is running, you should see an active status in green.

```
root@SOC-Automation-TheHive:~# systemctl start elasticsearch.service
root@SOC-Automation-TheHive:~# systemctl enable elastic.service
Failed to enable unit: Unit file elastic.service does not exist.
root@SOC-Automation-TheHive:~# systemctl enable elasticsearch.service
root@SOC-Automation-TheHive:~# systemctl status elasticsearch.service
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; vendor preset: enabled)
   Active: active (running) since Wed 2026-09-23 14:28:40 UTC; 1min 10s ago
     Docs: https://www.elastic.co/guide/en/elasticsearch/reference/8.10/index.html
   Main PID: 49529 (java)
    Tasks: 142 (limit: 19037)
  Memory: 8.4G (peak: 8.4G)
    CPU: 16min 10.627s
   CGroup: /system.slice/elasticsearch.service
           └─49529 /usr/share/elasticsearch/jdk/bin/java
             └─49592 /usr/share/elasticsearch/jdk/bin/java
               └─49620 /usr/share/elasticsearch/modules/analysis-icu/lib/icu4j.jar

Sep 23 14:28:20 SOC-Automation-TheHive systemd[1]: Started elasticsearch.service.
Sep 23 14:28:40 SOC-Automation-TheHive systemd[1]: Started elasticsearch.service.

root@SOC-Automation-TheHive:~# systemctl start thehive
root@SOC-Automation-TheHive:~# systemctl enable thehive
root@SOC-Automation-TheHive:~# systemctl status cassandra.service
● cassandra.service - LSB: distributed storage system
   Loaded: loaded (/etc/init.d/cassandra; generated by SysV)
   Active: active (running) since Thu 2026-09-24 10:00:00 UTC; 1min 10s ago
     Docs: man:systemd-sysv-generator(8)
```

7.4.4 Set a user with the right permissions for TheHive

- Create a thehive user.

```
sudo /usr/share/elasticsearch/bin/elasticsearch-users useradd  
thehive -p <thehive_user_password> -r superuser
```

- Replace <thehive_user_password> with a secure password you choose for your TheHive user.

7.5 Install and configure TheHive

TheHive packages are distributed as RPM and DEB files, as well as ZIP binary packages, all available for direct download via tools like wget or curl, with installation performed manually. All packages are hosted on an HTTPS-secured website and come with a SHA256 checksum and a GPG signature for verification.

Destination path

The commands below use /tmp/ as the download path. Replace it with the full local directory path where you want to save the files. Download the installation package along with its SHA256 checksum and signature files.

- Using wget:

```
wget -O /tmp/thehive_5.8.0-1_all.deb
```

```
https://thehive.download.strangebee.com/5.8/deb/thehive\_5.8.0-1\_all.deb
```

```
wget -O /tmp/thehive_5.8.0-1_all.deb.sha256
```

```
https://thehive.download.strangebee.com/5.8/sha256/thehive\_5.8.0-1\_all.deb.sha256
```

```
wget -O /tmp/thehive_5.8.0-1_all.deb.asc
```

```
https://thehive.download.strangebee.com/5.8/asc/thehive\_5.8.0-1\_all.deb.asc
```

```
root@SOC-Automation-TheHive:/# wget -O /tmp/strangebee.gpg https://keys.download.strangebee.com/latest/gpg/strangebee.gpg
--2026-09-24 11:14:02-- https://keys.download.strangebee.com/latest/gpg/strangebee.gpg
Resolving keys.download.strangebee.com (keys.download.strangebee.com)...
0, ...
```

```
root@SOC-Automation-TheHive:/# wget -O /tmp/<file_name>.deb https://thehive.download.strangebee.com/<major.minor_version>/deb/<file_name>.deb
wget -O /tmp/<file_name>.deb.sha256 https://thehive.download.strangebee.com/<major.minor_version>/sha256/<file_name>.deb.sha256
wget -O /tmp/<file_name>.deb.asc https://thehive.download.strangebee.com/<major.minor_version>/asc/<file_name>.deb.asc
-bash: file_name: No such file or directory
-bash: file_name: No such file or directory
-bash: file name: No such file or directory
```

```

root@SOC-Automation-TheHive:/# wget -O /tmp/thehive_5.8.0-1_all.deb https://thehive.download.strangebee.com/5.8/deb/thehive_5.8.0-1_all.deb
wget -O /tmp/thehive_5.8.0-1_all.deb.sha256 https://thehive.download.strangebee.com/5.8/sha256/thehive_5.8.0-1_all.deb.sha256
wget -O /tmp/thehive_5.8.0-1_all.deb.asc https://thehive.download.strangebee.com/5.8/asc/thehive_5.8.0-1_all.deb.asc
--2026-09-24 11:12:56-- https://thehive.download.strangebee.com/5.8/deb/thehive_5.8.0-1_all.deb
Resolving thehive.download.strangebee.com (thehive.download.strangebee.com)...
Connecting to thehive.download.strangebee.com (thehive.download.strangebee.com)|:443... connected
HTTP request sent, awaiting response... 200 OK
Length: 362877692 (346M) [application/x-debian-package]
Saving to: '/tmp/thehive_5.8.0-1_all.deb'

```

- Using curl:

```
curl -o /tmp/thehive_5.8.0-1_all.deb
```

https://thehive.download.strangebee.com/5.8/deb/thehive_5.8.0-1_all.deb

```
curl -o /tmp/thehive_5.8.0-1_all.deb.sha256
```

https://thehive.download.strangebee.com/5.8/sha256/thehive_5.8.0-1_all.deb.sha256

```
curl -o /tmp/thehive_5.8.0-1_all.deb.asc
```

https://thehive.download.strangebee.com/5.8/asc/thehive_5.8.0-1_all.deb.asc

```

root@SOC-Automation-TheHive:/# curl -o /tmp/thehive_5.8.0-1_all.deb https://thehive.download.strangebee.com/5.8/deb/thehive_5.8.0-1_all.deb
curl -o /tmp/thehive_5.8.0-1_all.deb.sha256 https://thehive.download.strangebee.com/5.8/sha256/thehive_5.8.0-1_all.deb.sha256
curl -o /tmp/thehive_5.8.0-1_all.deb.asc https://thehive.download.strangebee.com/5.8/asc/thehive_5.8.0-1_all.deb.asc

```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
0	0	0	0	0	0	0	0

```

root@SOC-Automation-TheHive:/# curl -o /tmp/strangebee.gpg https://keys.download.strangebee.com/latest/gpg/strangebee.gpg

```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
0	0	0	0	0	0	0	0
100	100	7033	100	3179	100	3179	7027

7.6 Configure TheHive

- Start TheHive service and enable it at boot.

```
sudo systemctl start thehive
```

```
sudo systemctl enable thehive
```

- Verify that TheHive is running.

```
sudo systemctl status thehive
```

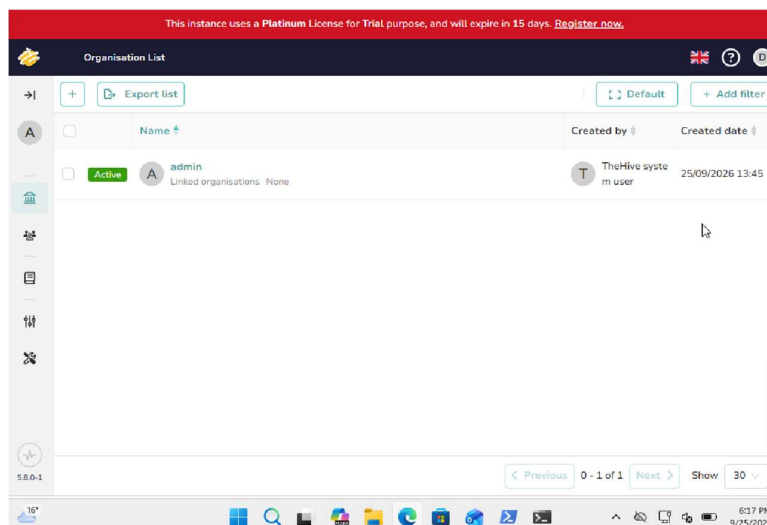
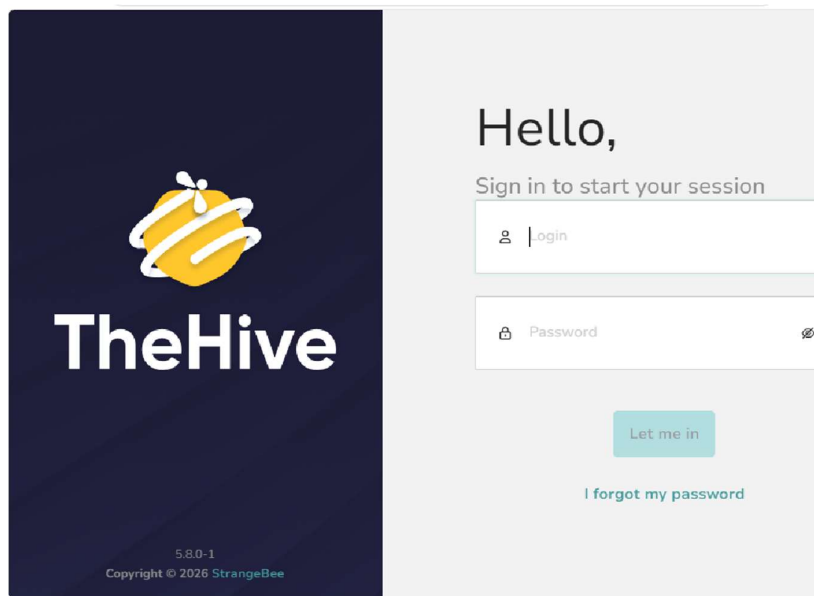
```

root@SOC-Automation-TheHive:~# systemctl status thehive
● thehive.service - Scalable, Open Source and Free S>
   Loaded: loaded (/usr/lib/systemd/system/thehive>
   Active: active (running) since Fri 2026-09-25 1>
      Docs: https://strangebee.com
    Main PID: 92640 (java)
      Tasks: 142 (limit: 19037)
     Memory: 1.2G (peak: 1.2G)
        CPU: 3min 24.005s
      CGroup: /system.slice/thehive.service
              └─92640 java -Dfile.encoding=UTF-8 -Dco>

Sep 25 11:44:55 SOC-Automation-TheHive systemd[1]: S>

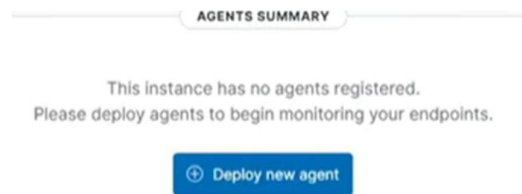
```

- If TheHive is running, you should see an active status in green.
- Go to the url <http://<IP address of TheHive instance>:9000>
- Default id is admin@thehive.local and password is secret



8. Deploying a new agent on Wazuh

- Click on “deploy new agent”



- Select windows
- Add your Wazuh instance IP address in server address

Deploy new agent

1 Select the package to download and install on your system:

LINUX

☐ RPM amd64 ☐ RPM aarch64

☐ DEB amd64 ☐ DEB aarch64

WINDOWS

☐ MSI 32/64 bits

macOS

☐ Intel

☐ Apple silicon

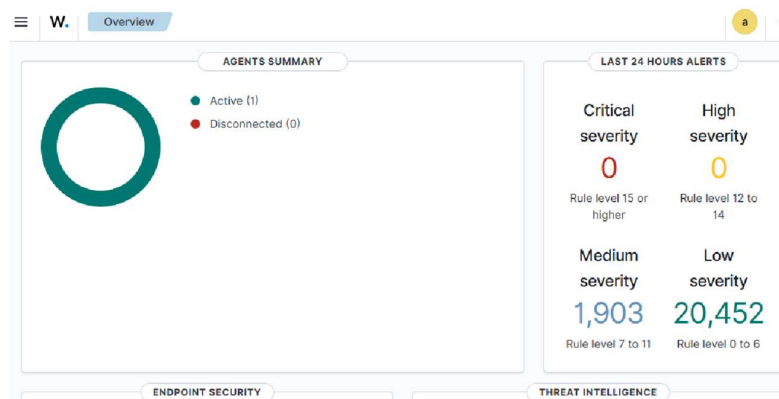
For additional systems and architectures, please check our [documentation](#).

2 Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

- Download the new agent using powershell. The command to install the new agent is provided below at the 4th step
- Once installed start it by using the below command and then refresh wazuh to view the newly installed agent:
net start wazuhsvc

```
PS C:\WINDOWS\system32> net start wazuhsvc
The requested service has already been started.
```



9. Windows Configuration

- On Windows 11 VM, open File explorer -> navigate to C drive -> Program files (x86) -> ossec-agent -> create a copy of ossec.conf -> open ossec.conf on notepad -> under "logical analysis",

Change: Application

To: <full name of Sysmon operational>

- To find the full name of Sysmon operational, open Event Viewer -> Applications and Services logs -> Microsoft Windows -> Sysmon -> right click on Operational -> click on Properties -> copy the full name of Sysmon Operational and paste it in ossec.conf
- Open Services -> right click on Wazuh -> Restart
- Disable Microsoft Defender: Search for Microsoft Security -> Navigate to Virus and Threat protection -> click on Manage Settings -> turn off Real-time Protection, Cloud Delivered Protection, Automatic Sample Submission and Tamper Protection

10. Downloading mimikatz_trunk.zip

- Head to <https://github.com/gentilkiwi/mimikatz/releases> and download [mimikatz_trunk.zip](#) and extract its contents
- Open a powershell, navigate to the extracted mimikatz_trunk file and type the following commands:

```
PS C:\Users\Sue\Downloads\mimikatz_trunk> cd x64
PS C:\Users\Sue\Downloads\mimikatz_trunk\x64> .\mimikatz.exe

.#####.  mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
.## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
## / \ ##  /*** Benjamin DELPY 'gentilkiwi' ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v #'    Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'    > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz # |
```

11. Changing ossec.conf Configuration

- Open powershell and log into your wazuh instance and create a copy of ossec.conf

```
root@SOC-Automation-Wazuh:~# cp /var/ossec/etc/ossec.conf ~/ossec_copy.conf
root@SOC-Automation-Wazuh:~# nano /var/ossec/etc/ossec.conf
```

- open ossec.conf and do the following changes as shown in the below screenshot

```
<alerts_log>yes</alerts_log>
<logall>yes</logall>
<logall_json>yes</logall_json>
```

- Restart wazuh

```
root@SOC-Automation-Wazuh:~# cd /var/ossec/logs/archives
root@SOC-Automation-Wazuh:/var/ossec/logs/archives# systemctl restart wazuh-manager
root@SOC-Automation-Wazuh:/var/ossec/logs/archives# systemctl restart wazuh-indexer
root@SOC-Automation-Wazuh:/var/ossec/logs/archives# systemctl restart wazuh-dashboard
root@SOC-Automation-Wazuh:/var/ossec/logs/archives# ls -la
total 92
```

- Configure filebeat.yml. Under filebeat.modules -> Set archives to true
- Restart filebeat.services

```
root@SOC-Automation-Wazuh:/var/ossec/logs/archives# nano /etc/filebeat/filebeat.yml
root@SOC-Automation-Wazuh:/var/ossec/logs/archives# systemctl restart filebeat.service
root@SOC-Automation-Wazuh:/var/ossec/logs/archives#
```

12. Creating a custom rule on Wazuh

- On wazuh: Navigate to Dashboard Management -> Click on Index patterns
-> Click on Create index pattern -> follow the below screenshots

Create index pattern

An index pattern can match a single source, for example, `filebeat-4-3-22`, or **multiple** data sources, `filebeat-*`.

[Read documentation](#)

Step 1 of 2: Define an index pattern

Index pattern name

`wazuh-archives-*`

Next step >

Use an asterisk (*) to match multiple indices. Spaces and the characters `\`, `/`, `?`, `"`, `<`, `>`, `]` are not allowed.

☐ Include system and hidden indices

✓ Your index pattern matches 1 source.

`wazuh-archives-4.x-2026.09.29`

Index

Step 2 of 2: Configure settings

Specify settings for your `wazuh-archives-*` index pattern.

Select a primary time field for use with the global time filter.

Time field

Refresh

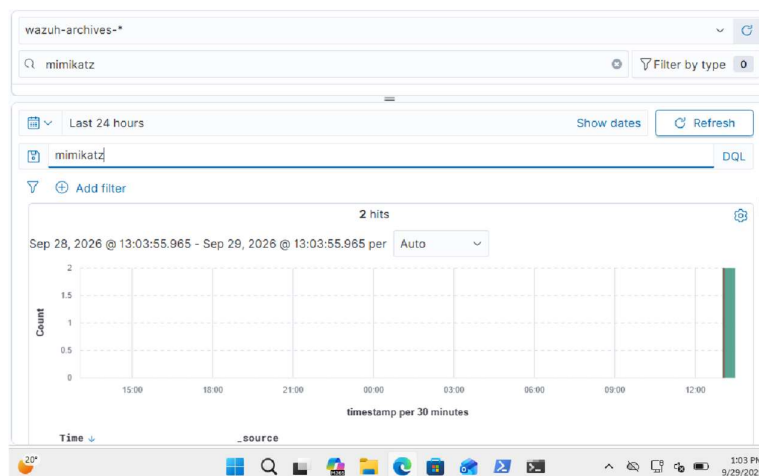
timestamp

> Show advanced settings

< Back

Create index pattern

- Execute mimikatz.exe in powershell
- Refresh Wazuh and navigate to Discover under explore, there you will be able to view the logs about mimikatz





- On Wazuh, navigate to Server Management -> Rules -> click on custom rules -> edit local_rules.xml

```
<rule id="100002" level="15">
  <if_group>sysmon_event</if_group>
  <field name="win.eventdata.originalFileName" type="pcre2">(?!mimikatz\\.exe</field>
  <description>Mimikatz Usage Detected</description>
  <mitre>
    <id>T1003</id>
  </mitre>
</rule>
```

- Open Microsoft Defender -> Go to Virus & Threat Protection -> Under Virus & Threat Protection Settings click on Manage Settings -> Click on Add or Remove Exclusions under Exclusions -> Click on Add an exclusion -> Select folders and exclude the entire C drive
- Execute mimikatz on powershell again after adding the new rule

```
PS C:\Users\Sue\Downloads\mimikatz_trunk\x64> ls

Directory: C:\Users\Sue\Downloads\mimikatz_trunk\x64

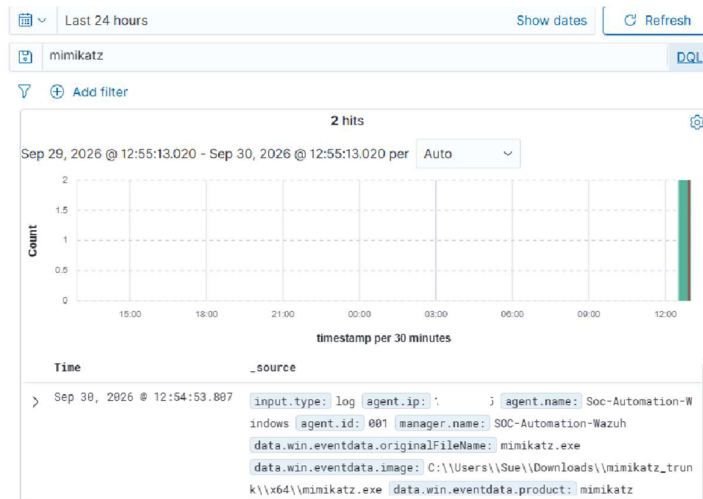
Mode                LastWriteTime         Length Name
----                -
-a----             9/30/2026 12:54 PM          1355264 mimikatz.exe
-a----             9/30/2026 12:54 PM
-a----             9/30/2026 12:54 PM
-a----             9/30/2026 12:54 PM

PS C:\Users\Sue\Downloads\mimikatz_trunk\x64> .\mimikatz.exe

.#####.  mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
.## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
## / \ ##  /** Benjamin DELPY 'gentilkiwi' ( benjamin@gentilkiwi.com )
## \ / ##   > https://blog.gentilkiwi.com/mimikatz
'## v #'    Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'   > https://pingcastle.com / https://mysmartlogon.com **/

mimikatz #
```

- Refresh Wazuh and navigate to Discover under explore, there you will be able to view the logs about mimikatz

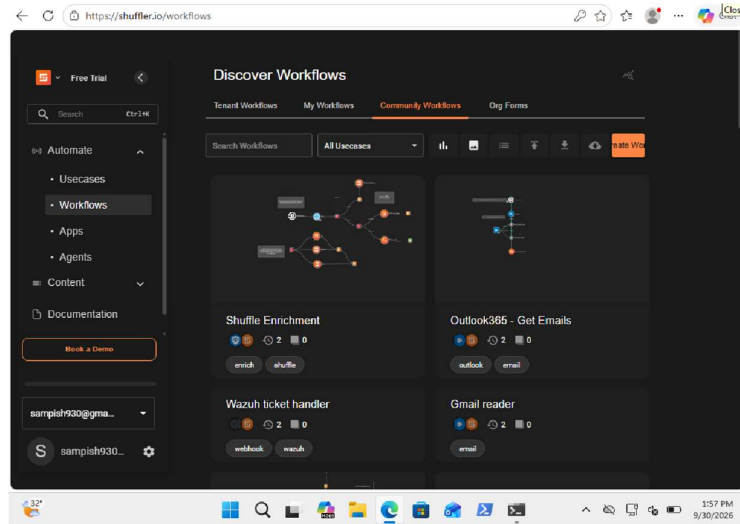


- You can even check that the newly added rule “Mimikatz Usage Detection” has been triggered

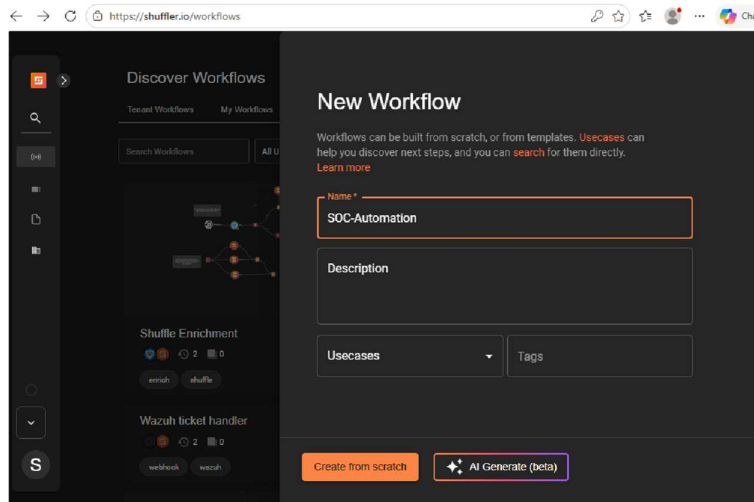


13. Integrating Wazuh with Shuffle

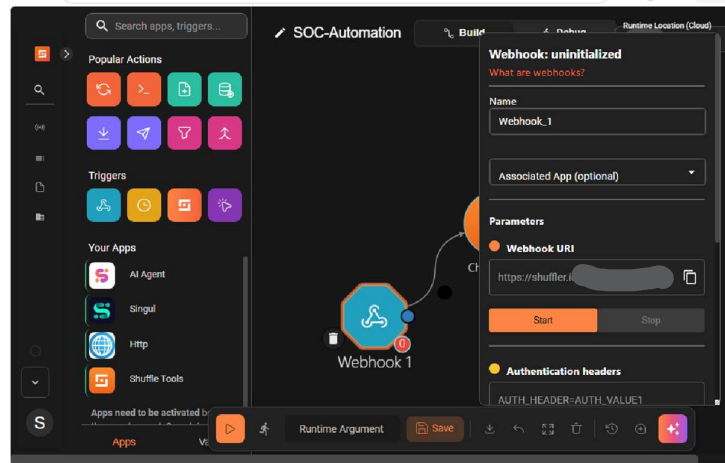
- Go to <https://shuffler.io> and sign in
- Navigate to workflows



- Click on create workflow



- After clicking on creating from scratch, attach a webhook and copy the webhook URI



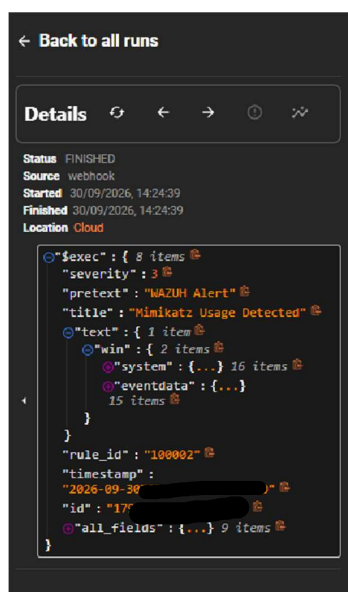
- Now we will configure ossec.conf again, in order to make it point to this URI
- Add the below in ossec.conf, save it and restart wazuh manager

```
<update_check>yes</update_check>
</global>

<integration>
  <hook_url>https://shuffler.io/api/v1</hook_url>
  <rule_id>100002</rule_id>
  <alert_format>json</alert_format>
</integration>

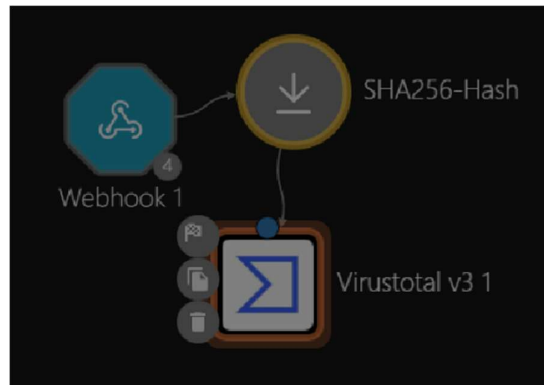
<alerts>
  <log_alert_level>3</log_alert_level>
```

- Go back to shuffle and start the webhook so that it can listen
- On powershell, execute mimikatz
- On shuffle, the logs of executing mimikatz can be seen indicating the webhook is working fine

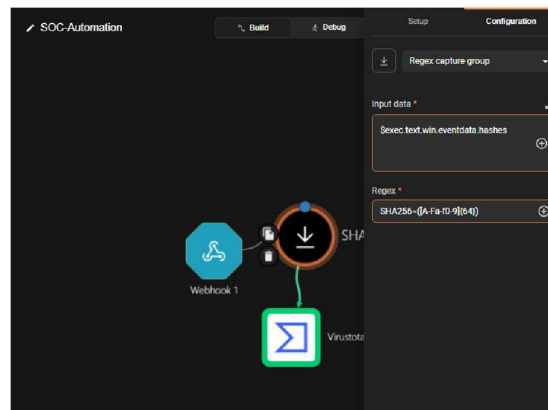


14. Creating a shuffle playbook with email integration and testing Mimikatz on it

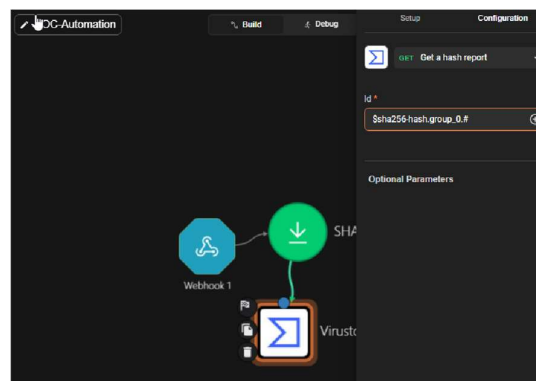
- Creating a playbook in shuffle using webhooks and virustotal and configuring it



- Configurations for SHA256-Hash

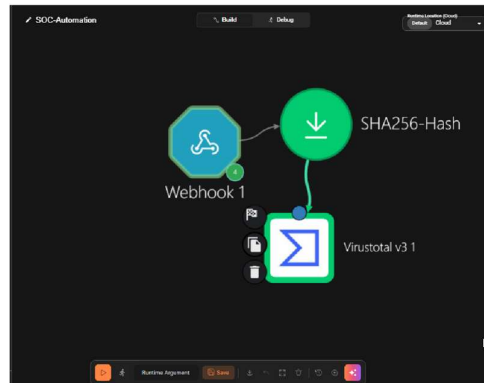


- Configurations for VirusTotal v3 1 after authentication



- Once you have created the Shuffle playbook, open powershell as an administrator and execute mimikatz

- After a few seconds you will notice, SHA256-Hash webhook has changed its colour from orange/grey/blue to green



- One can view the logs of mimikatz activity in shuffle debug section

This block contains three screenshots from the SOC-Automation interface. The top-left screenshot shows the 'All Workflow Runs' section with a table of runs. The top-right screenshot shows the 'Details' view for a specific run, displaying a JSON log entry. The bottom-left screenshot shows the 'Virusotal v3 1' step's debug logs, and the bottom-right screenshot shows the 'SHA256-Hash' step's debug logs.

All Workflow Runs

Run ID	Source	Status	Started	Finished	Location
01/10/2026, 12:48:35	webhook	Finished	01/10/2026, 12:48:35	01/10/2026, 12:48:37	Cloud
01/10/2026, 12:48:33	webhook	Executing	01/10/2026, 12:48:33		Cloud

Details

```

{
  "severity": 3,
  "pretext": "MIMIKATZ Alert",
  "title": "Mimikatz Usage Detected",
  "text": {
    "win": {
      "system": {
        "eventdata": {
          "rule_id": "100002",
          "timestamp": "2026-10-01",
          "id": "1",
          "all_fields": {
            "sha256-hash": {
              "success": true,
              "group_0": {
                "found": true
              }
            }
          }
        }
      }
    }
  }
}

```

Virusotal v3 1

```

{
  "status": 200,
  "body": {
    "data": {
      "url": "https://www.virustotal.com",
      "headers": {
        "cookies": {}
      },
      "success": true
    }
  }
}

```

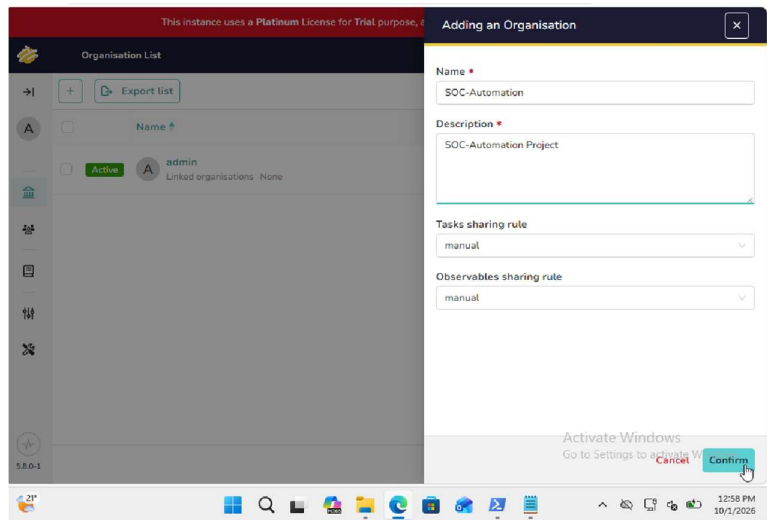
SHA256-Hash

```

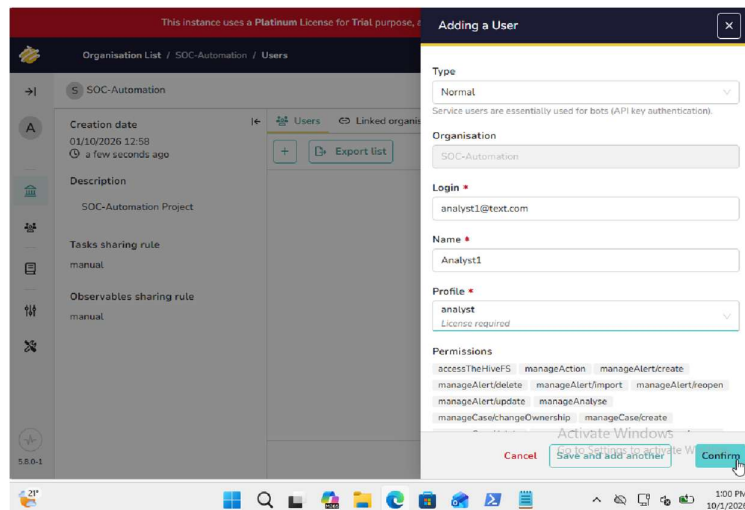
{
  "sha256-hash": {
    "success": true,
    "group_0": {
      "found": true
    }
  }
}

```

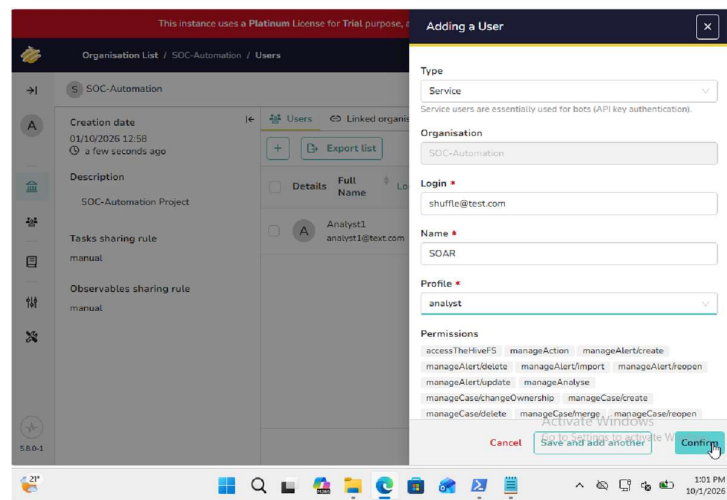
- Create an account in TheHive and click on it to create a new user



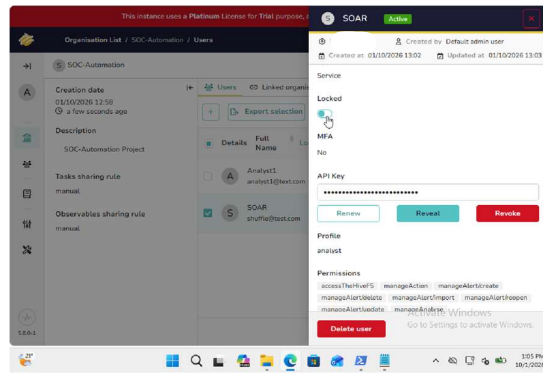
- Creating a new user named analyst1



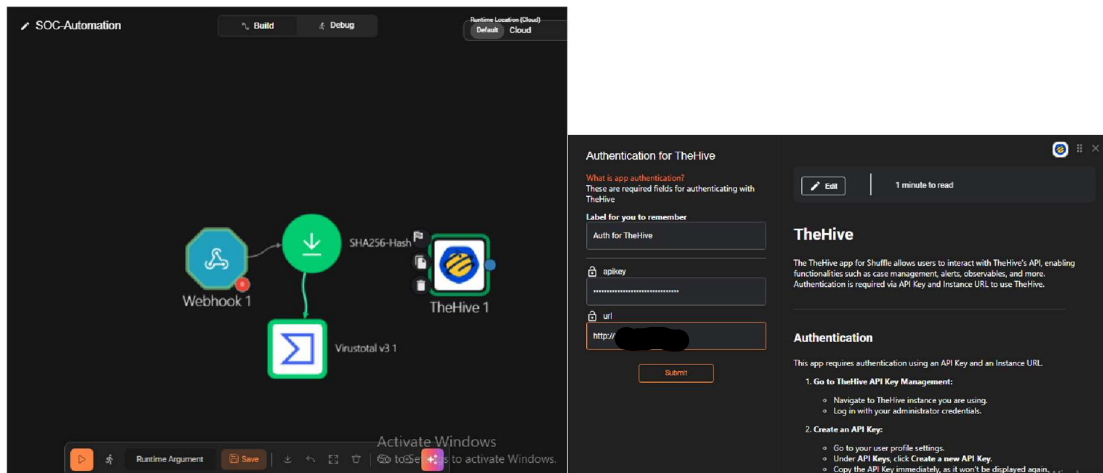
- Creating another user name SOAR



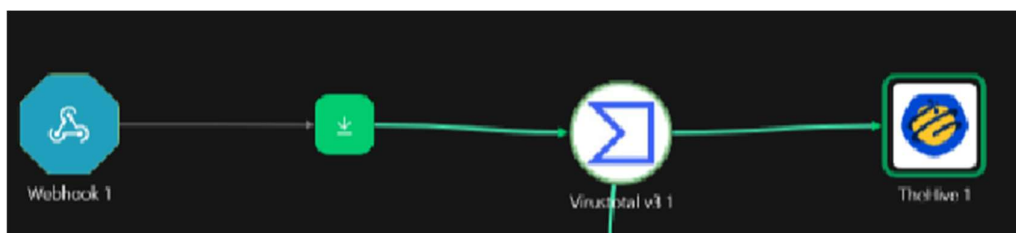
- Create an API key for username SOAR



- Add TheHive in the shuffle playbook and authenticate it with the generated API key



- Configure it's advanced setting and connect VirusTotal v3 1 to it



POST Create alert

Simple Advanced

Body *

```
{
  "description": "${exec.title}",
  "externallink": "${externallink}",
  "flag": false,
  "page": 2,
  "severity": "3",
}
```

Optional Parameters

- Execute mimikatz.exe in powershell and check if TheHive 1 is working

Status FINISHED

Source Parent Execution

Started 01/10/2026, 14:12:28

Finished 01/10/2026, 14:12:30

Location Cloud

```
{
  "$exec": { 8 items
    "severity": 3
    "pretext": "WAZUH Alert"
    "title": "Mimikatz Usage Detected"
    "text": { 1 item
      "win": { 2 items
        "system": {...} 16 items
        "eventdata": {...} 15 items
      }
    }
  }
  "rule_id": "100002"
  "timestamp": "2026-10-01T14:12:28.000Z"
  "id": "1"
  "all_fields": {...} 9 items
}
```

SHA256-Hash

regex_capture_group

```
{
  "$sha256-hash": { 2 items
    "success": true
    "reason": "CLEANED after finishing. Edit the 'Result cleanup' in the workflow edit menu to disable node cleanup..."
  }
}
```

Virustotal v3 1

get_a_hash_report

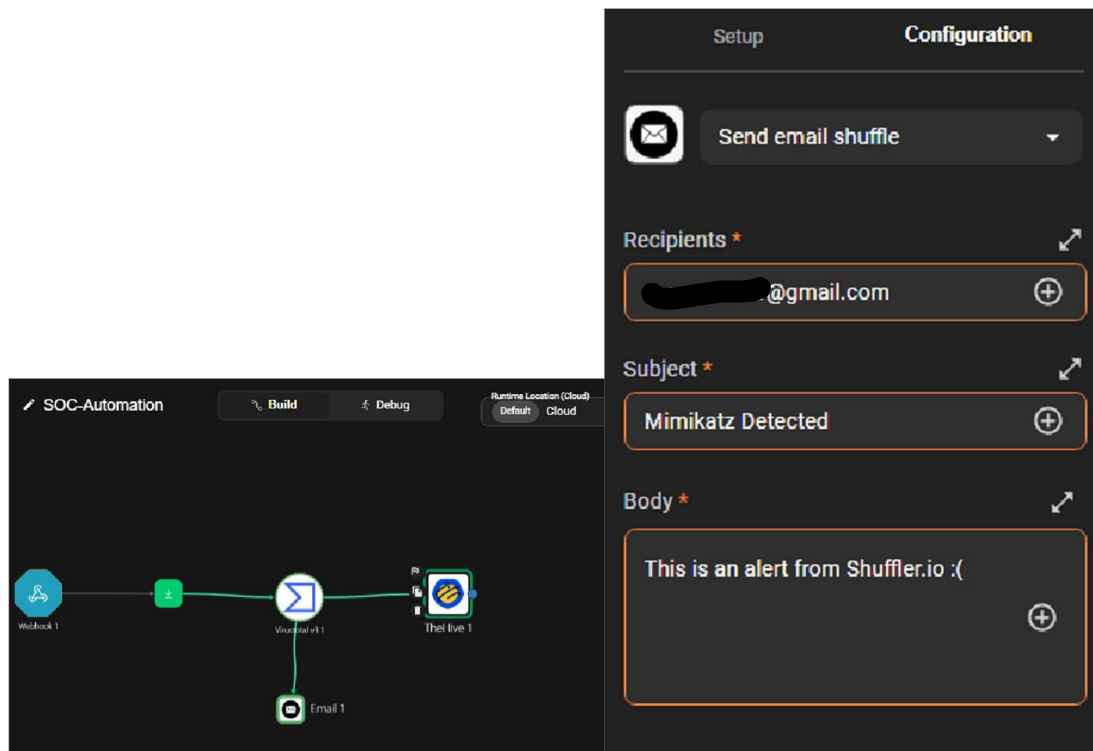
```
{
  "$virustotal_v3_1": { 2 items
    "success": true
    "reason": "CLEANED after finishing. Edit the 'Result cleanup' in the workflow edit menu to disable node cleanup..."
  }
}
```

TheHive 1

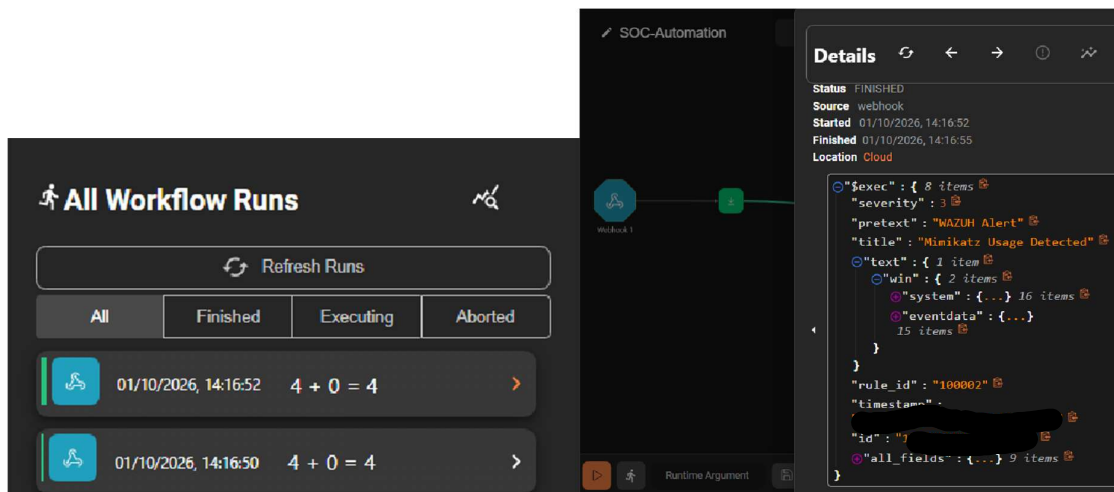
post_create_alert

```
{
  "$thehive_1": { 6 items
    "status": 201
    "body": {...} 26 items
    "url": "http://localhost:1234"
    "headers": {...} 10 items
    "cookies": {} 0 items
    "success": true
  }
}
```

- Attach email to VirusTotal v3 1 and configure it



- Execute mimikatz.exe in powershell
- An email will be sent to the given email



 **SHA256-Hash**
regex_capture_group

```
{
  "$sha256-hash": { 3 items
    "success": true
    "group_0": [...] 1 item
    "found": true
  }
}
```

 **VirusTotal v3 1**
get_a_hash_report_

```
{
  "$virustotal_v3_1": [ 1 item
    { 6 items
      "status": 200
      "body": { 1 item
        "data": { 4 items
          "id": [REDACTED]
          "type": "file"
          "links": { 1 item
            "self": "https://www.virustotal.com/analysis/[REDACTED]"
          }
          "attributes": {...}
        }
      }
    }
  ]
  "url": "https://www.virustotal.com/analysis/[REDACTED]"
}
```

```
{
  "https://www.virustotal.com/analysis/[REDACTED]"
  "headers": {...} 9 items
  "cookies": {} 0 items
  "success": true
}
```

 **email 1**
send_email_shuffle

```
{
  "$email_1": { 2 items
    "success": true
    "reason": "Email was successfully sent"
  }
}
```

 **TheHive 1**
post_create_alert

```
{
  "$thehive_1": { 6 items
    "status": 201
    "body": {...} 26 items
    "url": "http://[REDACTED]/api/v1/alert"
    "headers": {...} 10 items
    "cookies": {} 0 items
    "success": true
  }
}
```

Sent by email app: Mimikatz Detected Inbox x



Shuffle Email App <email-app@shuffler.io>
to me ▾

2:32 PM (0 minutes ago)

This is an alert from Shuffler.io :(

Sent by email app: Mimikatz Detected

Inbox x



Shuffle Email App <email-app@shuffler.io>

2:32 PM (0 minutes ago)



to me

This is

from: **Shuffle Email App** <email-app@shuffler.io>

to: @gmail.com

date: Oct 1, 2026, 2:32 PM

subject: Sent by email app: Mimikatz Detected

mailed-by: em6406.shuffler.io

signed-by: shuffler.io

security: Standard encryption (TLS) [Learn more](#)



Shuff

to me

...

tes ago)

